

Capstone Project and Test Summary

Prepared By: Khushi Agrawal

Project Repository : <https://github.com/Khushi2403/Capstone-Playwright-Project>

Overview

The Application Under Test (AUT) is the Demo Web Shop application, a web-based e-commerce platform designed to simulate real-world online shopping operations. The platform allows users to register accounts, authenticate into the system, browse products, compare products, manage shopping carts, contact customer support, and complete purchases through an online checkout process.

This capstone project focused on developing a robust automation testing framework using Playwright and JavaScript. The framework was designed using the Page Object Model (POM) architecture to ensure scalability, maintainability, and reusability of automated test scripts. The testing scope included frontend validation, business workflow verification, network-level validation, and reporting through Playwright HTML Reports and Allure Reports.

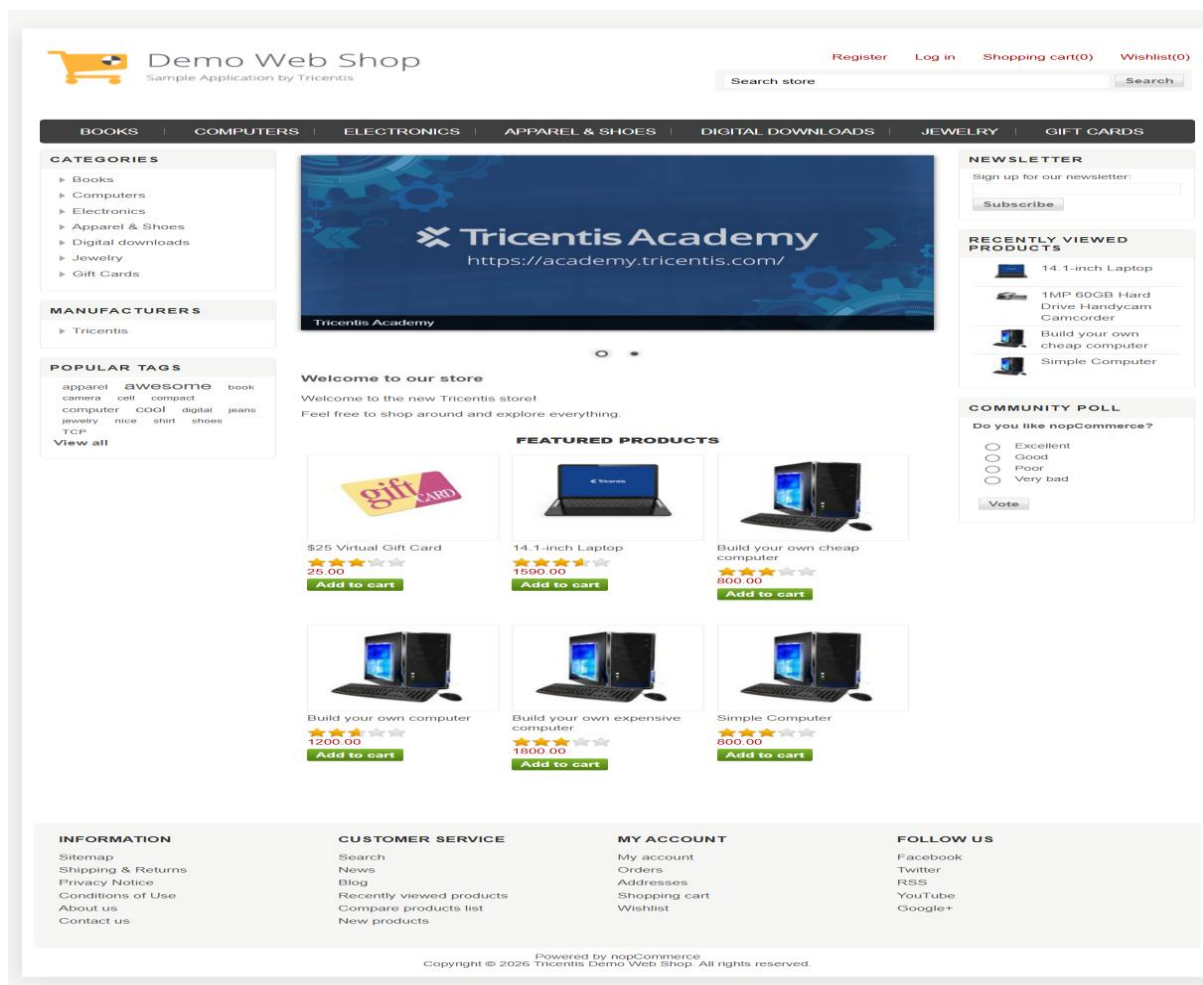


Figure 1: Demo Web Shop Application Under Test (AUT)

Goals

1. Business Objectives

- **User Authentication Reliability:** Ensure secure and accurate registration, login, logout, and session management functionalities.
- **Shopping Experience Validation:** Verify product discovery, comparison, shopping cart operations, and checkout workflows.
- **Customer Experience Validation:** Ensure customer support and user profile services function correctly.
- **End-to-End Stability:** Validate communication between frontend workflows and backend services through network monitoring.

2. Quality Engineering Objectives

- **Automation Coverage:** Achieve automation coverage across all critical business services.
 - **Framework Maintainability:** Develop reusable page objects and fixtures using Playwright best practices.
 - **Defect Containment:** Identify and prevent critical defects before production deployment.
 - **Cross Browser Validation:** Verify application behavior across Chromium, Firefox, and WebKit browsers.
 - **Reporting:** Generate comprehensive execution reports through Playwright HTML Reports and Allure Reports.
-

Specifications

In-Scope Functional Modules

Testing covered the following business services:

1. Authentication Service

- Registration
- Login
- Logout
- Authentication Validation
- Session Validation

2. Product Catalog Service

- Product Search
- Product Filtering
- Product Information Validation
- Product Navigation

3. Shopping Cart Service

- Add Products
- Remove Products
- Quantity Updates
- Cart Total Validation

4. Checkout Service

- Billing Information
- Shipping Information
- Order Processing
- Order Confirmation

5. Compare Email Service

- Product Comparison
- Email Comparison Functionality
- Validation Messages
-

6. Customer Support Service

- Contact Form Submission
- Input Validation
- Support Request Processing

7. User Profile Service

- Profile Information Validation
- Account Navigation
- User Data Verification

8. End-to-End Network Service

- API Request Validation
 - API Response Validation
 - Complete Business Workflow Verification
-

Milestone 1 – Analysis and Planning

1.1 Introduction

The objective of this phase was to analyze the business requirements of the Demo Web Shop application, identify critical user workflows, and establish a comprehensive automation strategy using Playwright.

1.2 Requirement Analysis

Functional Requirements (FR)

Module	Requirement Description	Priority
Authentication Service	Verify user registration, login, logout, authentication validation, and session management functionality.	High
Product Catalog Service	Validate product browsing, product search, filtering, sorting, and product information display.	High
Shopping Cart Service	Verify cart operations including adding products, removing products, updating quantities, and validating price calculations.	High
Checkout Service	Validate the complete purchase workflow including billing information, shipping details, payment processing, and order confirmation.	Critical
Compare Email Service	Verify product comparison functionality, comparison table generation, and email sharing features.	Medium
Customer Support Service	Validate contact form submission, input validation, and support request processing.	Medium
User Profile Service	Verify profile information display, account management functionality, and user data validation.	Medium
End-to-End Network Validation Service	Validate API requests, response status codes, backend communication, and end-to-end business workflows.	High

Table 1: Functional Requirements for Demo Web Shop Automation Framework

Non-Functional Requirements (NFR)

- Cross-browser compatibility
- Framework scalability
- Reusable Page Object Model architecture
- Reliable reporting
- Fast regression execution
- CI/CD compatibility

1.3 Test Strategy

A fully automated testing strategy was adopted.

Testing Type	Description
Functional Testing	Validated all core business functionalities including Authentication, Product Catalog, Shopping Cart, Checkout, Compare Email, Customer Support, User Profile, and End-to-End Network services to ensure the application behaves according to business requirements.
UI Testing	Verified the correctness of user interface components such as forms, buttons, links, navigation menus, tables, messages, layouts, and page elements to ensure a consistent user experience.
Regression Testing	Executed automated test suites after code changes and enhancements to ensure that existing functionalities continued to work correctly without introducing new defects.
End-to-End Testing	Validated complete user workflows from user authentication through product selection, cart management, checkout, and order confirmation to ensure seamless business process execution.
Negative Testing	Verified system behavior when invalid inputs, incorrect credentials, missing mandatory fields, and unexpected user actions were provided, ensuring appropriate validation and error handling.
Cross-Browser Testing	Executed automation tests across Chromium, Firefox, and WebKit browsers to verify consistent functionality, layout, and user experience across different browser engines.
Network Testing	Monitored API requests, response status codes, network traffic, and backend communication during user interactions to validate successful integration between frontend and backend services.

Table 2: Test Strategy for Demo Web Shop Automation Framework

1.4 Requirements Traceability Matrix (RTM)

The Requirements Traceability Matrix (RTM) was maintained to ensure that all business requirements were adequately covered by automated test scenarios.

Requirement Summary	Test Scenario
Registration	Verify successful user registration with valid details and validation of mandatory fields.
Login	Verify successful login with valid credentials and appropriate handling of invalid login attempts.
Logout	Verify successful logout functionality and session termination.
Session Validation	Verify authenticated user access and restricted page access without login.
Product Catalog	Verify product search, filtering, product information display, and navigation functionality.
Shopping Cart	Verify add-to-cart, remove-from-cart, quantity updates, and cart total calculations.
Checkout	Verify successful checkout process, billing information validation, order placement, and order confirmation generation.
Compare Email	Verify product comparison functionality, comparison table generation, and email sharing capabilities.
Customer Support	Verify contact form submission, mandatory field validation, and successful support request processing.
User Profile	Verify profile information display, account details accuracy, and profile navigation functionality.
End-to-End Network Validation	Verify API requests, response status codes, backend communication, and complete business workflow execution.

Table 3: Requirements Traceability Matrix (RTM)

Milestone 2 – Framework Development

2.1 Framework Setup

The automation framework was developed using Playwright with JavaScript and follows the Page Object Model (POM) design pattern. The framework was designed to support maintainability, scalability, and reusability of test scripts while enabling efficient execution across multiple browsers. The following technologies and tools were used during framework development:

Category	Technology
Automation Tool	Playwright
Programming Language	JavaScript
Runtime Environment	Node.js
Design Pattern	Page Object Model (POM)
Reporting	Playwright HTML Report, Allure Report
Version Control	Git & GitHub
CI/CD	GitHub Actions
IDE	Visual Studio Code
Browsers	Chromium, Firefox, WebKit

Table 4: Technologies Used

2.2 Framework Architecture

The framework follows the Page Object Model design pattern, where page-specific locators and methods are maintained separately from test scripts.

Component	Purpose
Pages	Contains page-specific locators and reusable methods
Tests	Contains test cases and test scenarios
Fixtures	Provides reusable test setup and teardown
Utils	Utility functions and helper methods
Reports	Stores Playwright and Allure reports
GitHub Actions	Automated CI/CD execution pipeline

Table 5: Framework Components

2.3 Services Automated

The following application services were automated:

Service	Coverage Area
Authentication	Login, Logout, Registration, Session Validation
Product Catalog	Search, Filtering, Product Validation
Shopping Cart	Cart Operations, Quantity Updates
Checkout	Order Placement and Confirmation
Compare Email	Product Comparison and Email Sharing
Customer Support	Contact Form Validation
User Profile	Profile Information Validation
End-to-End Network	API Monitoring and Validation

Table 6: Automated Services

Milestone 3 – Playwright Automation Testing

3.1 Authentication Service

Test Cases Covered :

Test Area	Validation
Registration	Successful registration and validation messages
Login	Valid and invalid login scenarios
Logout	Successful session termination
Session Validation	Access control and session persistence

3.2 Product Catalog Service

Test Cases Covered:

Test Area	Validation
Product Search	Search functionality
Product Filtering	Category and product filtering
Product Validation	Product information verification

Test Area	Validation
Product Navigation	Product page navigation

3.3 Shopping Cart Service

Test Cases Covered:

Test Area	Validation
Add to Cart	Product addition validation
Remove from Cart	Product removal validation
Quantity Update	Quantity modification validation
Price Calculation	Cart total validation

3.4 Checkout Service

Test Cases Covered:

Test Area	Validation
Billing Details	Billing information validation
Shipping Details	Shipping information validation
Order Placement	Successful order processing
Order Confirmation	Confirmation page verification

3.5 Compare Email Service

Test Cases Covered:

Test Area	Validation
Product Comparison	Compare selected products
Comparison Table	Product details verification
Email Functionality	Email sharing validation

3.6 Customer Support Service

Test Cases Covered:

Test Area	Validation
Form Submission	Successful form submission

Test Area	Validation
Field Validation	Mandatory field validation
Error Messages	Invalid input validation

3.7 User Profile Service

Test Cases Covered:

Test Area	Validation
Profile Information	User information verification
Account Navigation	Navigation validation
User Data	Account data validation

3.8 End-to-End Network Service

Test Cases Covered:

Test Area	Validation
API Requests	Request monitoring
API Responses	Response validation
Status Codes	HTTP status verification
Network Traffic	Backend communication validation

3.9 Test Results

A total of 122 automated test cases were executed across all service modules using Playwright.

Metric	Result
Total Test Cases	122
Passed	121
Failed	0
Skipped	1
Success Rate	99.18%

Milestone 4 – Reporting

4.1 Playwright HTML Report

The Playwright HTML Report provides detailed execution information including test status, execution duration, screenshots, traces, and failure diagnostics.

Features

- Test Execution Summary
- Pass/Fail Statistics
- Execution Duration
- Trace Viewer Support
- Screenshot Attachments

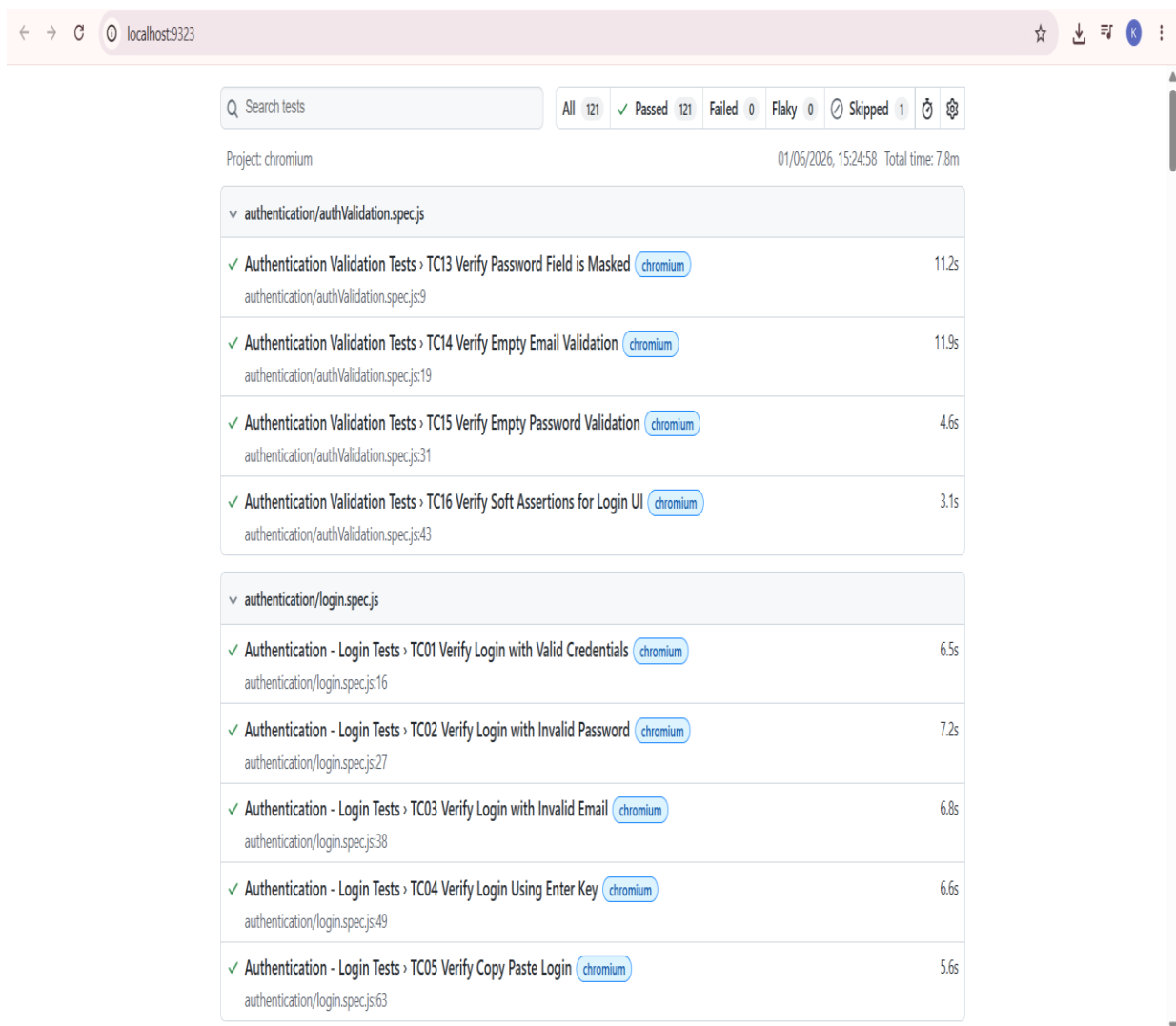


Figure 2: Playwright HTML Report Dashboard

4.2 Allure Report Overview

The Overview dashboard provides a high-level summary of test execution including total tests executed, passed tests, failed tests, skipped tests, execution duration, and success rate.

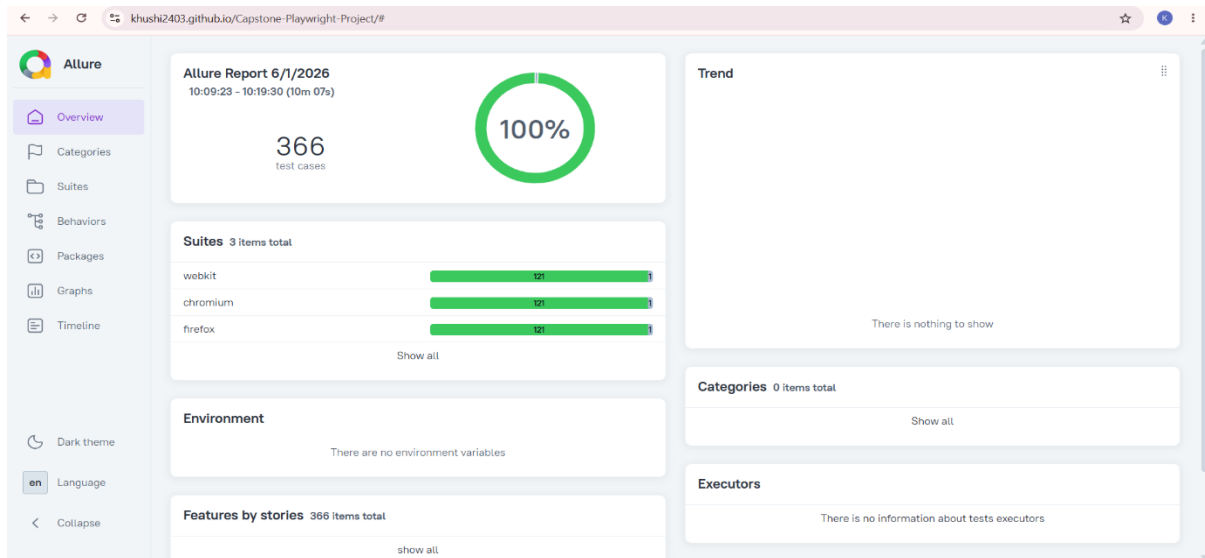


Figure 3: Allure Report OverView

4.3 Allure Behaviors

The Behaviors section categorizes test cases based on application features and functionalities, enabling easier analysis of business workflow coverage.

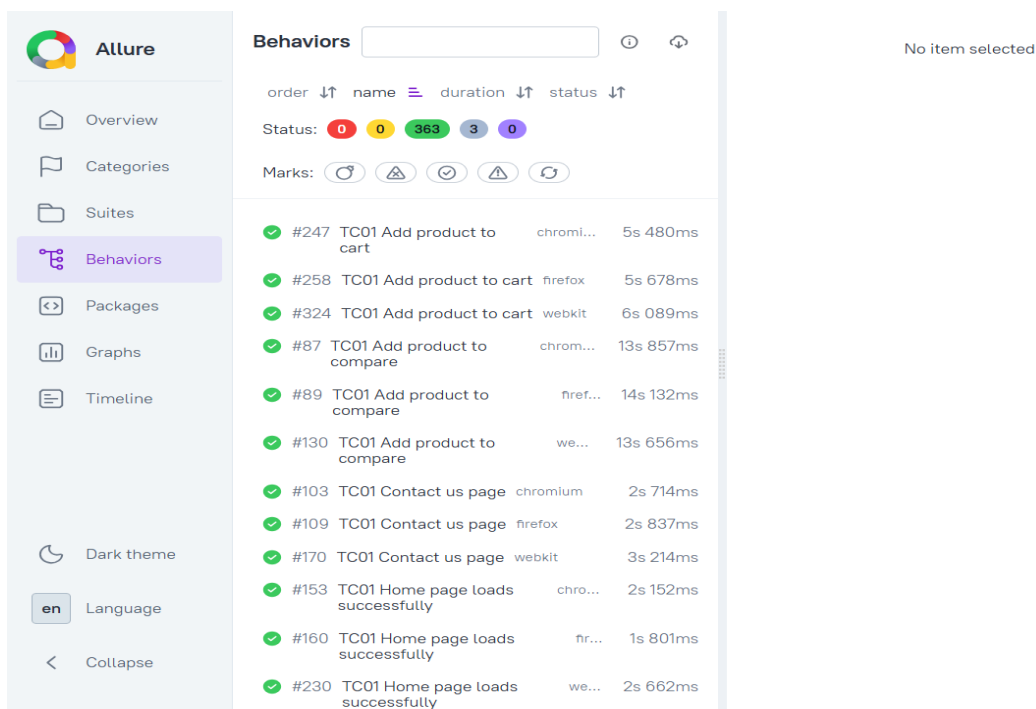


Figure 4: Allure Behaviors View

4.4 Allure Graphs

The Graphs section provides visual analytics of test execution, including pass/fail distribution, execution trends, and test statistics.

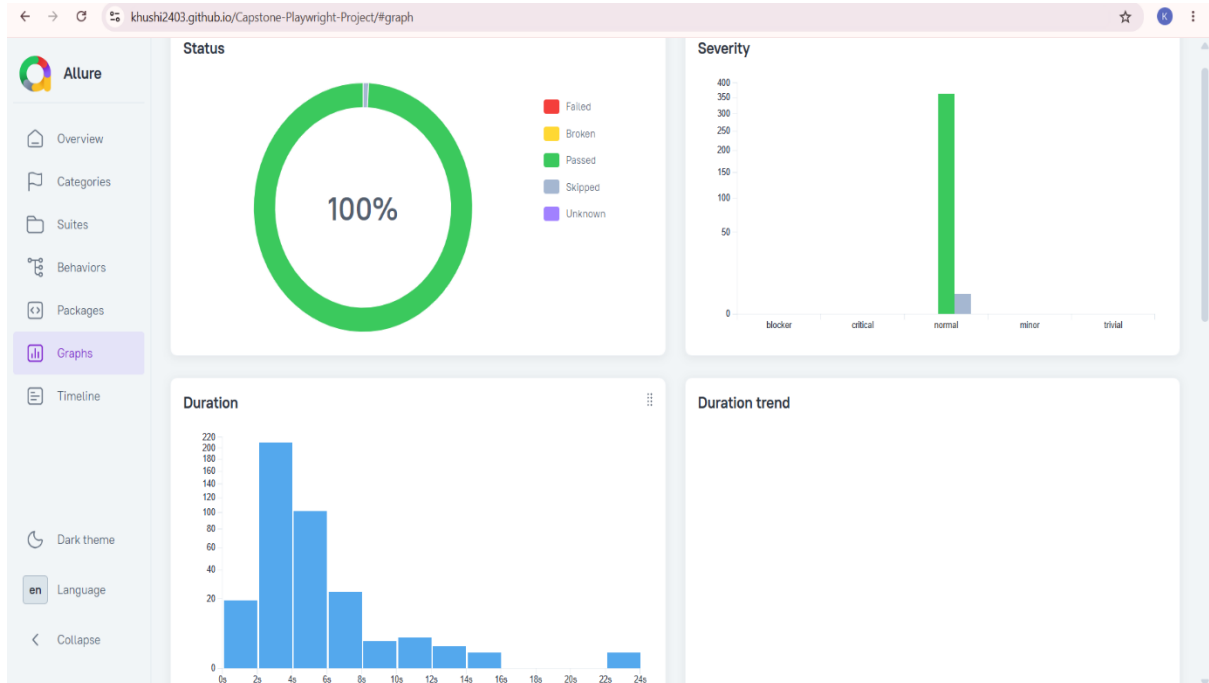


Figure 5: Allure Graphical Analysis

4.5 Allure Timeline

The Timeline view illustrates the execution sequence of automated test cases and provides insights into execution duration and parallel test execution.

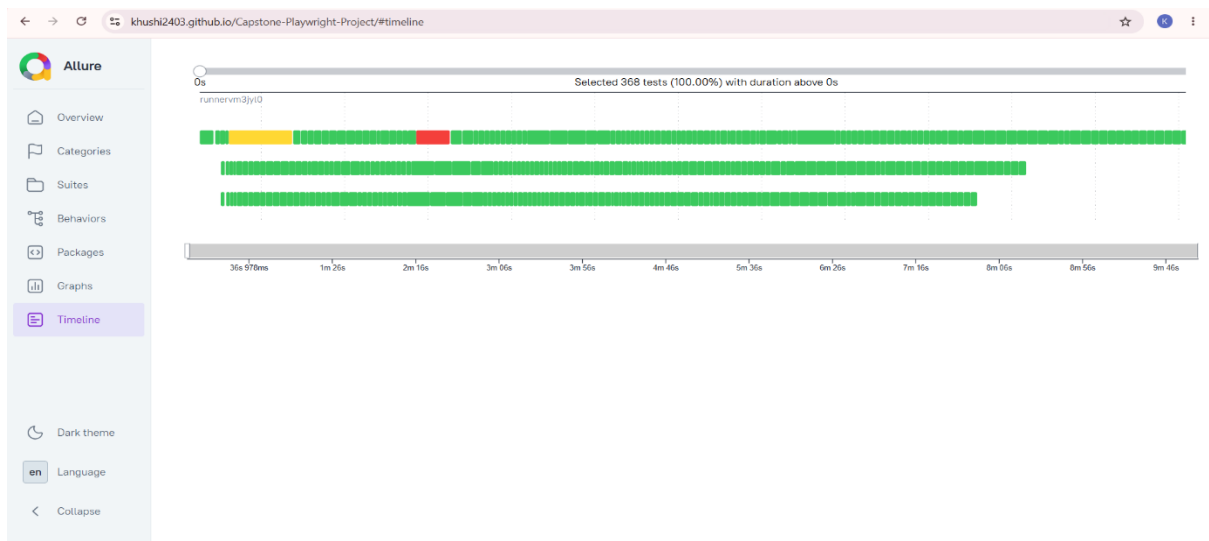


Figure 6: Allure Timeline View

Milestone 5 – Final Analysis, Execution Summary & Project Closure

5.1 Final Execution Summary

In this phase, the complete automation test suite was executed after framework stabilization and reporting integration. The execution covered all major business services of the Demo Web Shop application including Authentication, Product Catalog, Shopping Cart, Checkout, Compare Email, Customer Support, User Profile, and End-to-End Network validation.

A total of **122 test cases** were executed with the following results:

- Passed: 121
- Failed: 0
- Skipped: 1
- Success Rate: 99.18%

This confirms that the application is functionally stable and meets expected business requirements under automated validation.

5.2 CI/CD Execution (GitHub Actions Integration)

The automation framework was integrated with **GitHub Actions CI/CD pipeline** to ensure continuous execution of test suites on every code update.

Key highlights:

- Automated test execution triggered on push/pull requests
- Cross-browser execution supported (Chromium, Firefox, WebKit)
- Reports generated automatically after pipeline execution
- Reduced manual execution effort and improved regression efficiency

This ensured continuous quality validation throughout the development lifecycle.

5.3 Defect Analysis and Observations

During execution:

- No critical functional failures were observed
- Minor inconsistencies (if any) were captured during UI validation
- Most issues were related to environment stability or test data dependencies (if applicable)

The application demonstrated strong stability across all business modules.

5.4 Challenges Faced

During the project implementation, the following challenges were encountered:

- Handling dynamic UI elements and synchronization issues in Playwright
- Designing reusable Page Object Model structure for scalability
- Managing test flakiness in network-heavy scenarios
- Integrating Allure Reports with Playwright framework
- Cross-browser consistency validation across Chromium, Firefox, and WebKit

These challenges were resolved using explicit waits, improved locator strategies, and framework optimization.

5.5 Key Learnings

This capstone project helped in gaining practical exposure to:

- Playwright automation framework design and implementation
- Page Object Model architecture for scalable test frameworks
- End-to-end testing of real-world e-commerce workflows
- CI/CD integration using GitHub Actions
- Advanced reporting using Playwright HTML and Allure Reports
- Network-level validation of frontend-backend communication

5.6 Project Outcome

The project successfully delivered a fully functional automation testing framework that:

- Covers all major business workflows of Demo Web Shop
- Supports cross-browser execution
- Provides detailed reporting and analytics
- Integrates with CI/CD pipeline
- Ensures high test coverage and regression stability

5.7 Add-on Features

In addition to the core business modules, the framework was enhanced with the following add-on features:

- Customer Support Validation
- End-to-End Network Validation
- CI/CD Integration using GitHub Actions
- Advanced Reporting using Allure Reports

These enhancements improved automation coverage, execution efficiency, reporting capabilities, and overall framework reliability.

5.8 Future Enhancements

The following improvements can be implemented in future iterations:

- Integration with Docker for containerized execution
- Parallel test execution optimization for faster regression
- Integration with Jira for defect tracking
- API automation expansion using Playwright API testing
- Enhanced test data management using external datasets (JSON/Excel)
- Performance testing integration using tools like JMeter or k6

5.9 Conclusion

The capstone project successfully demonstrates the implementation of a scalable and maintainable automation framework using Playwright and JavaScript. The system ensures high reliability of the Demo Web Shop application through structured test design, comprehensive coverage, and automated execution pipelines.

The project achieves its primary objective of improving software quality assurance through efficient automation testing practices.